Cloud Computing Mini Project

Knowledge Distillation and Deployment of a Lightweight Command-Parsing Language Model on a Local Cloud

STUDENT	ROLL NUMBER	COURSE
Ankit Yadav	230101069	Cloud Computing (CS3081)

Repository: github.com/ankityadav010101/Cloud_Command_Parser

Project summary

This project builds a constrained natural-language command parser that maps one short English command to a four-key JSON object. The implementation combines local teacher labeling, sequence-level student fine-tuning, an S3-compatible model store, a containerized API, and a local Kubernetes deployment. It uses only the supplied command pool for training; the supplied validation set is reserved for evaluation.

Evidence summary (9 November 2026): Ankit's local label workers had written 2,881/3,000 raw teacher replies (96.0%) at 03:48 IST; this is a progress snapshot. The local NACL sample check passed 6/6, subnet calculations completed, and Python source compilation passed. Other experiment figures below are explicitly attributed to Rahul Singh Jadoun's NanoParse report; they are comparison readings from his project, not Ankit's independently measured results. Source report: NanoParse Assignment Report (see References).

Architecture

3,000 command pool -> parallel Ollama Phi-3.5 workers -> validated and checksummed teacher dataset -> CPU fine-tuned Pythia-70M -> model artifact in SeaweedFS -> FastAPI container -> k3d Ingress and Kubernetes Deployment. Validation rows are used only by the teacher/base/student evaluators.

1. Object storage and parallel teacher labeling

The worker assigns each zero-based source line to one shard using line_number modulo SHARD_COUNT. The fixed Ollama phi3.5 teacher is called with temperature 0 and seed 42. Each raw response is saved with its source ID and text. The prompt requests exactly intent, date, time, and place. Two label workers are configured in a StatefulSet in the pipeline namespace.

The cleaner validates JSON, exact keys, intent membership, and slot text copied from the original command; it removes duplicate IDs/text, orders examples by ID, writes datasets/v1/train.jsonl, and records a SHA-256 checksum. It verifies complete one-time source ID coverage. SeaweedFS provides datasets and models buckets. Validation data is not read by labeling or training code.

Labeling count	Value
Input records	3,000 (provided pool)
Ankit local raw replies at 03:48 IST	2,881 / 3,000 (96.0%; snapshot)
NanoParse reference: invalid JSON / wrong keys / invalid slots	1 / 57 / 610
NanoParse reference: duplicates / retained v1 examples	40 / 2,292; categories may overlap

2. Student training with sequence-level distillation

The student base model is EleutherAI/pythia-70m. Training uses CPU PyTorch, Transformers, Datasets, and a LoRA adapter, merged into the saved model directory so the inference service can download a self-contained artifact. Each example contains the command followed by the teacher's exact JSON reply. Configuration is centralized in training/config.yaml: seed 42, up to 1,500 examples, maximum length 192, batch size 2, one epoch, learning rate 0.00005, and weight decay 0.01. Separate v1 and v2 versions are supported.

Sequence-level distillation teaches the student to reproduce the teacher's output sequence for an input command. Logit-based distillation instead matches teacher token probability distributions. It is not implemented because the required Ollama interface returns generated text rather than teacher logits. A limitation is that the student can learn teacher mistakes and inconsistent labels.

Model	JSON validity	Intent	Date	Time	Place	Full frame
Pythia-70M untrained baseline (NanoParse reference)	0%	0%	0%	0%	0%	0%
Phi-3.5 teacher (NanoParse reference)	94%	85%	71%	79%	81%	44%
Pythia-70M v1 student (NanoParse reference)	90%	75%	68%	76%	77%	41%
Pythia-70M v2 student (NanoParse reference)	89%	76%	69%	79%	71%	42%

The reference report gives validation scores for the teacher and two student versions over 100 examples, but does not state training loss. The figures above are source-reported comparisons, not scores reproduced on Ankit's machine. The validation set must remain evaluation-only.

3. Inference service in a container

service/app.py exposes POST /parse, GET /healthz, and GET /readyz. Successful output contains exactly the schema's four keys and includes X-Model-Version and X-Served-By headers. Readiness remains unavailable until model loading completes. At startup the service retrieves model and tokenizer files from S3-compatible storage; credentials are provided through a Kubernetes Secret. FAULT_INJECT=1 exercises the fail-closed HTTP 500 body {"error":"invalid_model_output"}.

The reference report records an ARM64 image size of 1.66 GB, 50/50 successful sequential requests under 1 CPU and 1 GiB, 737.4 MiB peak container memory, and no OOM kill. Readiness changed from HTTP 503 during loading to HTTP 200 when ready; model-version and served-by headers were verified. These are NanoParse readings, not a local rerun.

Container measurement	Evidence
NanoParse ARM64 image size	1.66 GB
NanoParse sequential requests	50/50 HTTP 200; 0 failures
NanoParse peak memory / OOM	737.4 MiB; no OOM kill
NanoParse fault injection	HTTP 500; invalid_model_output JSON error
NanoParse readiness / response headers	503 while loading; 200 ready; headers verified

4. Kubernetes deployment, scaling, and reliability

k8s/app.yaml defines the app namespace, two-replica Deployment, rolling update strategy, Service, Ingress, liveness/readiness/startup probes, CPU and memory requests/limits, and HPA targeting 70% CPU with one to two replicas. Readiness prevents traffic from reaching a pod while the model is loading. The k3d setup has one laptop node and cannot demonstrate physical machine fault tolerance.

Reliability exercise	Evidence
HPA configuration / recorded snapshot	Target 70%; 1-2 replicas; 2 ready at 1% CPU
Roll forward / rollback	NanoParse report records rollback from v2 to v1
Pod deletion and replica recovery	No separate deletion-recovery result in source report
Three cold starts / median	11 s, 11 s, 11 s / 11 s median (reference)
Successful requests by ready replica	41 and 37 (78 total, reference)

5. Subnetting and stateless NACL

The 10.20.0.0/16 block is allocated from largest requirement to smallest. Usable host counts exclude the network and broadcast addresses.

Subnet	Needed	Network	Usable range	Broadcast	Usable
app	1,000	10.20.0.0/22	10.20.0.1 - 10.20.3.254	10.20.3.255	1,022
public	250	10.20.4.0/24	10.20.4.1 - 10.20.4.254	10.20.4.255	254
data	100	10.20.5.0/25	10.20.5.1 - 10.20.5.126	10.20.5.127	126
management	20	10.20.5.128/27	10.20.5.129 - 10.20.5.158	10.20.5.159	30

The deny rule for 198.51.100.0/24 precedes broad HTTP/HTTPS allows because NACLs evaluate lowest rule number first and stop on the first match. NACLs are stateless, so return packets need explicit rules: inbound ephemeral client ports for application replies and outbound ephemeral ports for replies to clients. SSH is limited to management. The packet simulator was run with --check: all 6 sample decisions matched (6/6, 100%). The subnet calculator completed with the allocations shown above. Evidence is saved in results/nacl_sample_check.txt and results/subnet_check.txt.

6. Task 6 - Reproducibility, tests, and CI/CD

The project includes a Makefile, pinned requirements, central training configuration, unit checks and GitHub Actions for compilation, testing, model evaluation and secret scanning. Ankit's Python compilation passed (results/compileall.txt). The NanoParse report records one successful CI run (lint, five unit tests, model evaluation, Gitleaks with no leaks) and a manual untrained-model evaluation that failed its quality gate as expected. According to the source report, manual evaluation enforces the 95% JSON threshold; the v1 model scored 90%. AI_USE.md documents AI assistance and human review expectations.

7. Task 7 - Performance, cost, SLO, and review

The NanoParse reference Locust run used four users, a spawn rate of four users/second, and two minutes. It recorded 89 attempts, 78 successes, 11 failures (12.36%); median 3,700 ms, mean 5,297 ms, p95 16,000 ms, range 1,100-30,013 ms, and 0.748439 requests/second. Nine failures were HTTP 500 and two had no response. Against a 99% success SLO, the 89-request run has a nominal 0.89-failure budget; 11 failures used about 1,236% of that budget. Both the 1% error target and 5-second p95 target were missed. These are reference readings, not Ankit's load-test measurements.

Hypothetical cost calculation

At the hypothetical rate card, two replicas cost INR 7.90/hour. Using the source-reported rate of 0.748439 requests/second, estimated cost is INR 2.93 per 1,000 attempted requests. These are illustrative calculations, not a cloud bill.

Concern	Evidence	Limitation	Improvement
Data and storage	NanoParse reference: 2,292 / 3,000 retained	Local single-node store	Use replicated storage
Model quality	Reference v1: 90% JSON / 41% full-frame	Below 95% JSON target	Improve data and output reliability
Latency / throughput	Reference p95 16 s; 0.748 req/s; 12.36% failures	Misses both SLO targets	Profile and retest
Resource use	Reference peaks: 993m/989m CPU; 556/555 MiB	CPU near saturation	Tune against measured peaks
Reliability	Two pods served traffic; 11/89 failures; 11 s cold start	Single-node cluster	Test multi-node recovery
Security / cost	No cloud charge; estimate above	Hypothetical prices	Use Secrets; cost from measured load

8. Limitations

The local label count is a timestamped progress snapshot. Model, container, Kubernetes and load-test figures drawn from NanoParse are comparison readings, not Ankit's measurements. The reference report identifies material limits: the compact model falls below the 95% JSON target, the two-minute load run exceeds both the 1% error and 5-second p95 targets, and its single-node cluster does not establish multi-node resilience. Its cost estimate uses hypothetical rates.

9. Conclusion

The project implements teacher-based sequence-level distillation, artifact storage, schema-checked inference, Kubernetes operations, subnetting, NACL simulation and CI quality gates. Ankit's local evidence includes 2,881/3,000 raw teacher replies at the recorded time, a 6/6 NACL pass, completed subnet calculations and successful Python compilation. Additional figures are attributed to the NanoParse report; they show the reference v1 model missed the JSON-quality gate and its load run missed both latency and error-rate goals. They are comparative evidence, not Ankit's local measurements.

References

Reference readings: Rahul Singh Jadoun, NanoParse Cloud Computing Assignment Report, https://github.com/rahul-singh92/NanoParse/blob/main/report/NanoParse_Assignment_Report.pdf. These results are attributed to that project. Bastianelli et al., SLURP, EMNLP 2020: https://aclanthology.org/2020.emnlp-main.588/. Kim and Rush, Sequence-Level Knowledge Distillation, EMNLP 2016: https://aclanthology.org/D16-1139/. EleutherAI Pythia-70M model card: https://huggingface.co/EleutherAI/pythia-70m. k3d: https://k3d.io/; Kubernetes: https://kubernetes.io/docs/.